

# A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

---

**Thai Nguyen Vu<sup>1</sup>**   **Dung Van Bui<sup>2</sup>**   **Nhut Tri Do<sup>2</sup>**   **Van Du Nguyen<sup>3</sup>**

SOICT 2026

<sup>1</sup>Campus in Ho Chi Minh City, University of Transport and Communications, Ho Chi Minh City, Vietnam

<sup>2</sup>University of Information Technology, VNU-HCM, Ho Chi Minh City, Vietnam

<sup>3</sup>Faculty of Information Technology, Nong Lam University, Ho Chi Minh City, Vietnam

# **I. PROBLEM AND CONTRIBUTIONS**

---

## Edge intrusion detection: three open gaps

Detectors now *fit* on single-board hardware — but a monolithic one classifies **every** flow and saturates an 8 GB ARM board.

### What the literature does not tell us

1. Everything is measured on a **single device**.
2. Throughput, latency and **energy** are never reported together.
3. Offline scores come from **leaky protocols**.

### Our question

How should inference be *placed* across a constrained two-node edge cluster?

### Our answer

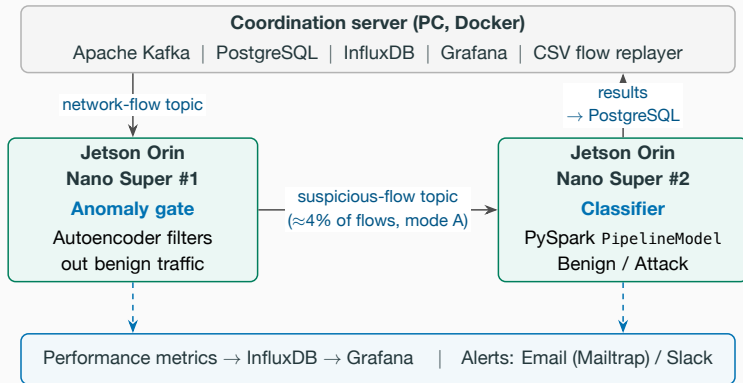
Split it: a cheap **autoencoder gate** on board #1, the heavy **PySpark classifier** on board #2, joined by a Kafka topic.

1. A **two-stage distributed IDS**: anomaly gate + Spark classifier, physically separated across two boards.
2. **Three deployment modes** for a two-node Jetson cluster — pipeline split, horizontal Kafka scaling, Spark standalone cluster.
3. An **end-to-end evaluation**: throughput, latency percentiles, per-node CPU/RAM *and* energy per verdict.
4. **Leakage-aware provenance** — the deployed model is exactly the one our companion study selected under a leakage-free protocol.

## **II. SYSTEM AND EVALUATION SETUP**

---

# Split-deployment architecture



The gate's MSE threshold is calibrated *offline* on a held-out benign validation split — committed before evaluation, never tuned on the test set.

CICIDS2017: Sharafaldin et al., ICISSP 2018 · Kafka: Kreps et al., NetDB 2011 · Spark: Zaharia et al., HotCloud 2010.

# Three deployment modes, one workload

**Single node** full pipeline in one process — the reference.

**A: Pipeline split** gate on #1, classifier on #2 (our design).

**B: Horizontal** both boards run the *full* role, sharing a Kafka consumer group.

**C: Spark cluster** host as master, both boards as workers.

Throughput is counted as **final verdicts per second** — a flow is decided at the gate *or* at the classifier — so forwarded flows are never double-counted.

## Setup

2× Jetson Orin Nano Super  
(6-core A78AE, 8 GB,  $\approx 67$  TOPS)  
PySpark 3.4.1, Kafka 3.5, Wi-Fi LAN  
CICIDS2017 replay at 100 flows/s  
 $\geq 3$  repetitions, warmup excluded  
Energy via tegrastats, idle subtracted

### **III. RESULTS AND CONCLUSION**

---

## Result 1: splitting the pipeline buys 24× throughput

| Mode             | Verdicts/s  | p95 (ms)    | J / verdict |
|------------------|-------------|-------------|-------------|
| Single node      | 2.6         | 12.3        | 2.49        |
| <b>A: Split</b>  | <b>62.5</b> | <b>13.1</b> | <b>0.18</b> |
| B: Horizontal    | 5.9         | 21.6        | 2.25        |
| C: Spark cluster | 90.0        | 23.9        | 0.12        |

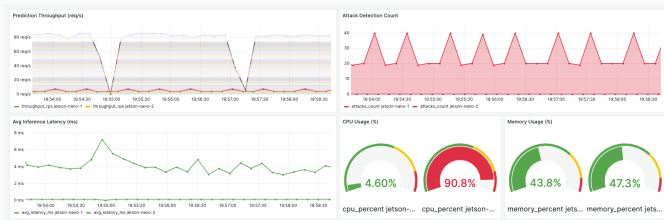
**Verdict:** the final benign/attack decision for one flow.

**p95:** 95% of flows get their verdict faster than this.

- **Split: 24× throughput, same p95.** The gate settles 95.8% of flows before Spark.
- **Horizontal: only 2.3×** — every board still runs Spark.
- Cluster: faster on average but **unstable** ( $\pm 25.6$ ).

**Pipeline split is the recommended default:** highest throughput with the smallest run-to-run spread.

## Result 2: the load asymmetry is visible live



At peak load, split mode:

- gate node: **4.6% CPU**
- classifier node: **90.8% CPU**

The classifier dominates the compute budget — which is exactly why moving it off the ingest path pays.

## Result 3: what the gate lets through

| Gate threshold<br>(benign quantile) | Attacks<br>forwarded | False<br>alarms | Flows<br>skipped |
|-------------------------------------|----------------------|-----------------|------------------|
| 0.99                                | 74.2%                | 1.01%           | 88.0%            |
| <b>0.995 (deployed)</b>             | <b>72.3%</b>         | <b>0.53%</b>    | 88.7%            |
| 0.999                               | 58.1%                | 0.10%           | 91.2%            |

Offline, on the CICIDS2017 test split; threshold fixed on benign validation data only.

- A flow the gate skips is final: **about 28% of attack flows** never reach the classifier at the deployed threshold.
- Lowering the threshold forwards more attacks but sends more traffic to Spark.
- The operator picks the point that fits the budget.

The throughput gain is paid for in recall: the gate is a **tunable trade-off**, not a free filter.

## Result 4: the price of one unified stack

We serve with Spark for *consistency with distributed training*. What does that cost? Same forest, same 29 deployed features, same board:

| Engine              | Flows/s       | p95 (ms)   | RAM (%) | mJ per flow |
|---------------------|---------------|------------|---------|-------------|
| PySpark (deployed)  | 22.6          | 532        | 24.3    | 71.6        |
| scikit-learn        | 204.4         | 49.8       | 13.3    | 2.91        |
| <b>ONNX Runtime</b> | <b>7870.8</b> | <b>1.1</b> | 15.7    | <b>0.06</b> |

mJ per flow: energy above the board's idle power.

ONNX Runtime sustains **348×** the deployed PySpark throughput. We report this as a *target for the next iteration* — export to ONNX/TensorRT for serving; keep Spark for training.

## What we showed

- A reproducible distributed edge IDS: Kafka + PySpark on a Jetson Orin Nano Super cluster.
- Splitting gate and classifier across boards: **2.6** → **62.5 flows/s** at p95  $\approx 13$  ms, gate skipping 95.8%.
- Energy per verdict: **2.5 J** → **0.18 J**.

## Limitations

- Two-node lab cluster over Wi-Fi; replayed traffic.
- Gate forwards 72% of attacks at the deployed threshold.
- GPU unused (CPU-only pipeline); evasion untested.

## Next

- ONNX/TensorRT serving; live traffic; retraining under drift.

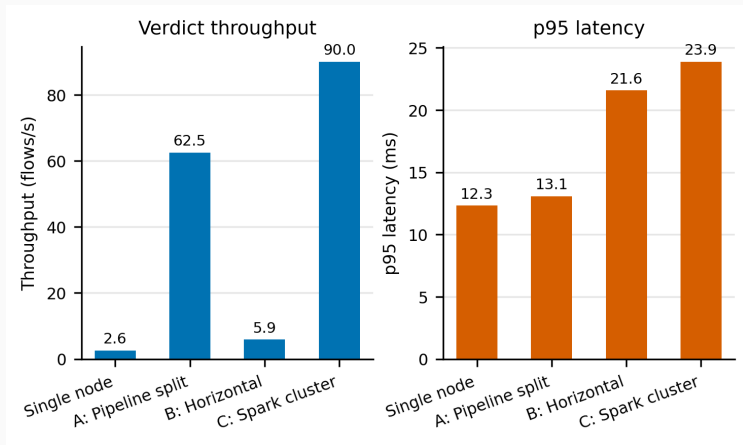
**Thank you — questions?**

## Key references

- [1] Sharafaldin, I., Lashkari, A. H. & Ghorbani, A. A. *Toward generating a new intrusion detection dataset and intrusion traffic characterization*. ICISSP 2018. **CICIDS2017**
- [2] Shi, W. et al. *Edge computing: Vision and challenges*. IEEE Internet of Things Journal 3(5), 2016. **edge setting**
- [3] Chandola, V., Banerjee, A. & Kumar, V. *Anomaly detection: A survey*. ACM Computing Surveys 41(3), 2009. **anomaly gate**
- [4] Lundberg, S. M. & Lee, S.-I. *A unified approach to interpreting model predictions*. NeurIPS 2017. **SHAP Top-30**
- [5] Zaharia, M. et al. *Spark: Cluster computing with working sets*. HotCloud 2010. **Spark**
- [6] Kreps, J., Narkhede, N. & Rao, J. *Kafka: a Distributed Messaging System for Log Processing*. NetDB 2011. **Kafka**
- [7] Mohan Kiran Kumar, K. et al. *Distributed Intrusion Detection System using Kafka and Spark Streaming*. ICVADV 2025. **closest system**
- [8] Arp, D. et al. *Dos and Don'ts of Machine Learning in Computer Security*. USENIX Security 2022. **leakage-aware protocol**

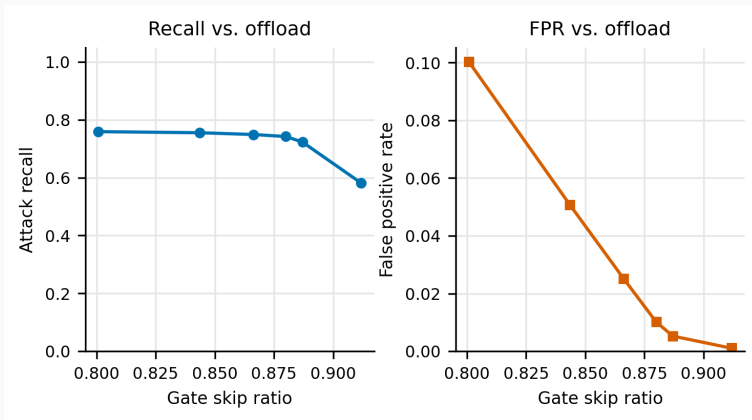
**Backup slides**

## Backup: deployment-mode comparison chart



Verdict throughput (left) and p95 latency (right).

## Backup: the gate is an operating curve, not a point



Attack recall (left) and false-positive rate (right) vs. gate-skip ratio. Raising the skip ratio offloads more Spark work and costs recall; the operator chooses the point.

## Backup: which model, and why

| Configuration                | F1     | Features | Size (MB)          | Edge rationale                    |
|------------------------------|--------|----------|--------------------|-----------------------------------|
| Baseline + XGBoost           | 0.9971 | 77       | 2.32               | Reference accuracy                |
| <b>SHAP Top-30</b> + XGBoost | 0.9970 | 30       | 0.003 <sup>†</sup> | Explainable; deployed feature set |
| RF Top-30 + XGBoost          | 0.9922 | 30       | 2.52               | Embedded ranking                  |
| PCA $k=40$ + XGBoost         | 0.9939 | 40       | 3.82               | Unsupervised reduction            |

We deploy **SHAP Top-30 with Random Forest** (offline F1 0.9955): a native Spark ML estimator, so the exported `PipelineModel` loads on ARM64 without the native libraries Spark's XGBoost integration needs. The input vector has **29** features — `destination_port` is in the SHAP ranking but excluded as label-leaking.

<sup>†</sup>Distributed saving under-reports size; true size  $\approx 2.5$  MB.

# Backup: full monitoring dashboard



Per-node throughput, detected attacks, inference latency, CPU/RAM for both boards,  
PostgreSQL-backed alerts.

## Backup: measurement protocol

- Host-side orchestration starts pipeline roles on the Jetsons over SSH, drives the load, collects per-node logs.
- $\geq 3$  repetitions per configuration; warmup traffic *excluded* from the measured window (no JVM/JIT inflation of early batches).
- Metric queries aligned to the exact load window, not a trailing range.
- Latency percentiles per node over *raw per-flow* samples; cluster p95 is conservatively the *worst node's* p95.
- End-to-end send-to-verdict latency from producer timestamps under NTP-synchronized clocks (excludes DB-write cost).
- Energy: tegrastats, 30 s idle baseline then load window; two-node modes sum both boards.